

Malicious Data Classification Using Structural Information and Behavioral Specifications in Executables

Sandeep Kumar

Department of Computer Science and Engineering
NITTTR Chandigarh, India
Sandeep.kaushal008@gmail.com

Naveen Aggarwal

Department of Computer Science and Engineering
UIET, Punjab University Chandigarh, India
navagg@gmail.com

C. Rama Krishna

Department of Computer Science and Engineering
NITTTR Chandigarh, India
Rkc_97@yahoo.com

Rakesh Sehgal

Department of Cyber Security Technology Development
CDAC Mohali, India
rks@cdac.in

Saurabh Chamotra

Department of Cyber Security Technology Development
CDAC Mohali, India
Saurabh@cdac.in

Abstract - With the rise in the underground Internet economy, automated malicious programs popularly known as malwares have become a major threat to computers and information systems connected to the internet. Properties such as self healing, self hiding and ability to deceive the security devices make these software hard to detect and mitigate. Therefore, the detection and the mitigation of such malicious software is a major challenge for researchers and security personals. The conventional systems for the detection and mitigation of such threats are mostly signature based systems. Major drawback of such systems are their inability to detect malware samples for which there is no signature available in their signature database. Such malwares are known as zero day malware. Moreover, more and more malware writers uses obfuscation technology such as polymorphic and metamorphic, packing, encryption, to avoid being detected by antivirus. Therefore, the traditional signature based detection system is neither effective nor efficient for the detection of zero-day malware. Hence to improve the effectiveness and efficiency of malware detection system we are using classification method based on structural information and behavioral specifications. In this paper we have used both static and dynamic analysis approaches. In static analysis we are extracting the features of an executable file followed by classification. In dynamic analysis we are taking the traces of executable files using NtTrace within controlled atmosphere. Experimental results obtained from our algorithm indicate that our proposed algorithm is effective in extracting malicious behavior of executables. Further it can also be used to detect malware variants.

Keyword- *malware detection; behavioral specifications; static analysis; dynamic analysis; system calls; classification algorithms.*

I. INTRODUCTION

Malware, short for malicious software, is a relatively new concept, but it is not a new research field. Intrinsically malware is a variety of hostile, intrusive or annoying software or program code designed to secretly access a computer system without the owner's informed consent. The art of programming has evolved into a profession concerned with how best to maximize the quality of software and how to create it [3]. From its very beginning in the 1960s, malware has also evolved into the most significant threat to computer network systems, especially in the last three decades. Along with the growth of the Internet, there has been a dramatic growth in instances of malware in recent years [10]. Threats on the Internet today is malware and the underlying root cause of most Internet security problems is malware. Malware makers are often looking for one-time development of specific code to generate new variants of existing malware [3], instead of developing new malware from scratch. In this case, variants of existing malware can be developed easily and quickly. Therefore, can be rapidly brought to market in the shadow economy [1]. This means there is a very big opportunity that any new malicious program found in the wild is a variation of a previous program. The increasing trend of generating malware variants disables antivirus companies to identify new

malware. This has been seen that by using manual process a large number of samples remain undetected or there is a significant gap between the malware release and detection using traditional antivirus software. So there is a need to develop automatic analysis technique to check this rapid malware generation trend.

Researcher have used static analysis approaches [4] to detect zero day malware. In static analysis the features are extracted using a PE file extractor API [9] but this approach fails when a file is packed or encrypted because entropy of some of packed benign is same as that of malicious file [6]. Hence a classification algorithm gives false alarm in this case. There are many cases where code reordering obfuscation is applied to a code variant of malware [3]. Similarly a sequence of NOP instruction has been included in the code to modify the syntactic signature of the variant but keep the semantics unchanged [10]. By inserting and retrieving dummy values through the stack or memory can not change the behavior of the program but makes difficult for reverse engineering.

There are many Advance detection techniques such as malicious code analyzer and semantic aware malware detection to know the obfuscation techniques of attackers by using behavioral specifications of malicious program. These approaches focus on to specify the general behavior exhibited by the entire family of malware instead of a particular instances. These specifications include self-unpacking and self propagation via email [2]. The main strength of these approach is to use high-level specification that abstract details specific to a malware instance. Thus obfuscation transformations techniques [3] in which behavior of malware is preserve but its syntax is changed are no longer effective for evading detection. These high level specification of malicious behavior used by these detectors are currently manually developed .It is a time consuming task and requires expert knowledge. To address this limitation, this paper presents an algorithm to automatically derive the behavioral specifications of malware by taking the trace in a controlled environment. These specification can be used by a malware. Since this algorithm also provides a succinct description of the malicious behavior, this can also be used by security expert for malware understanding. The rest of this paper is organized as follows: the related work is discussed in section II. Section III proposed our method including data collection, feature extraction, feature selection and machine learning techniques. An empirical

evaluation of the method is given in section IV and finally section V concludes this article.

II. RELATED WORK

Basically malware analyst uses two common approaches to analyze malware sample i.e Static and dynamic analysis. Static method is the most popular approach to identify whether a file is a malicious or benign file. Static analysis is based on the fact that every PE file has its own structural information [4]. This structural information is different for benign as well as malicious file. It has been observed experimentally that there is a false alarm in the case of packed benign [4]. We have suggested an algorithm where the packed benign is separated from unpacked benign using PEID tools. This has been observed that syntax of malware can be changed by obfuscation techniques such as polymorphism and metamorphism [3]. Hence existing detectors fails to detect different variants of the same malware. Additionally, static analysis is generally safer than dynamic analysis as the source code is not actually executed . We are using PECOFF4J jar file to extract static features from the binary file. In dynamic analysis many authors have suggested different approaches to extract the behavior specifications of malware.

Shafiq et al. [4] have proposed in 2009 PE-Miner: Mining Structural Information to Detect Malicious Executables in Real-time. In this author presented a real-time PE-Miner framework that automatically extracts unique features from portable executables to detect zero-day malware. This technique gives false alarm in the case of polymorphic and metamorphic malware. Jha et al. [11] have proposed in 2007, Mining Specifications of Malicious Behavior. In this author discussed that Malware detectors require a specification of malicious behavior. In this approach authors have calculated a dependency graph between the argument values of the system calls of malware file. Salehil et al. [9] have proposed in 2012, A Miner for Malware Detection Based on API Function Calls and Their Arguments. In this author describe that since signature based methods cannot identify sophisticated malware quickly and effectively, research is moving toward using runtime behaviors of benign and malicious files.

After the critical review, the authors have drawn the following inferences:

(a) Static analysis of malware fails in the case of polymorphic and metamorphic malware; (b) Malware authors can change the syntax of malware but semantics remains unchanged; (c) Using Dynamic

analysis we can detect polymorphic and metamorphic malware.

Based on these inferences, the problem definition and solution are explained in section III.

III. PROBLEM STATEMENT AND DESIGN METHODOLOGY

From the literature review, it is clear that there is a need to develop a classification system based on structural information and behavioral specifications of executables. In the same reference authors have proposed a scalable method using API call argument values to obtain higher accuracies. Feature selection techniques are applied to reduce the number of features and enhance the detection rates. Since API calls and the argument values of API calls could be a good representatives of binary behavior. We have used an approaches to find the argument values from the NtTrace system calls and based on them malicious behavior has been classified using J48 algorithms. To meet the objectives, we propose and implement prototype systems that characterize the inherent features of portable executable file and analyze them for quick and accurate analysis. In this paper we have proposed methods and techniques of classification, feature extraction and finding the relations among system calls from their traces. To complete this task the methodology flow chart is shown in Fig 1.

3.1 Proposed Architecture of Classification System

3.1.1 ClamAV Scanning System- ClamAV is an open source (GPL) antivirus engine designed for detecting Trojans, viruses, malware and other malicious threats. In this proposed ClamAV Scanning System, an effort has been made to load the data from a directory and by clicking on scan button ClamScan command will execute. It will detect the file, if its signature is already stored in the ClamAV database otherwise it will not detect. This file may be malicious or benign. To check whether this file is benign or malicious, a classification system has been developed. If it is malicious its signature will be loaded into ClamAV database.

3.1.2 Dynamic Analysis using NtTrace Execution Environment- We have proposed the following architecture for specifying malicious behavior in terms of dependences between system calls. We have implemented our own algorithm to find these relations between system calls [5],[11],[9], total 17 important relations has been shown in section IV in which total 12 relations having high frequency has been selected for classification .To make the network

connection autonomous Vidalia Bundle has been included in the proposed architecture, Fig 2.

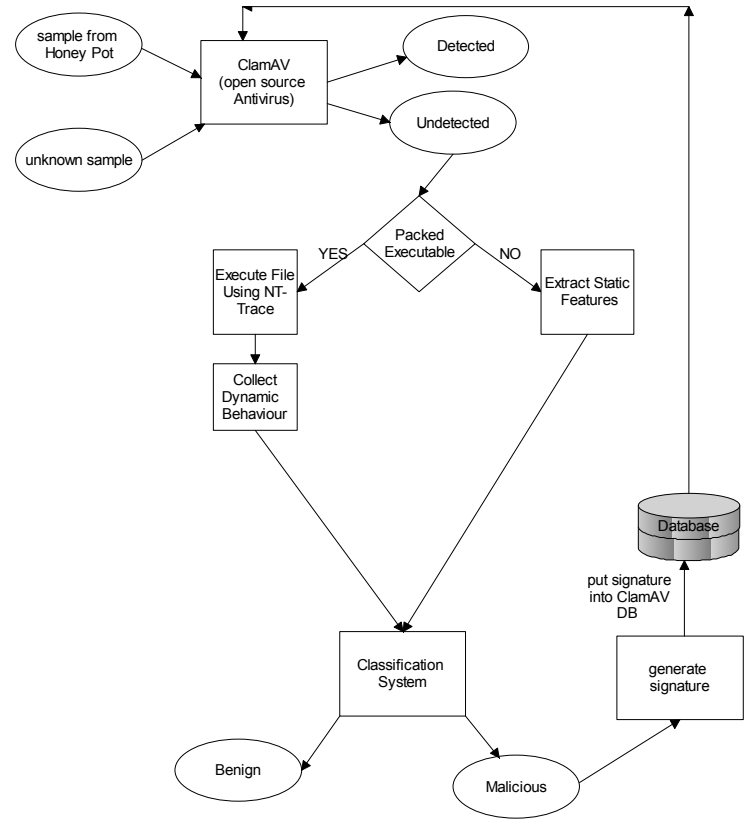


Fig. 1. Methodology Flow Chart

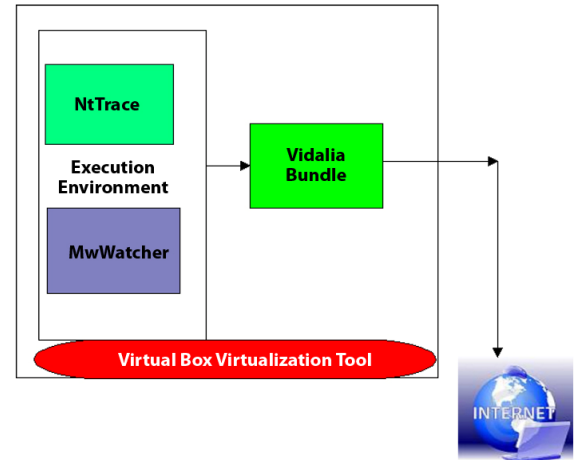


Fig. 2. Proposed Architecture for finding the execution Trace of Malware

The module is divided into 4 parts-

- Collecting Execution traces
- Finding the relationships between system calls.

- (c) Applying feature selection to use them for classification.
- (d) To calculate the performance matrices.

```

1.  INPUT : A set of malware and a set of benign
      programs.
2.  OUTPUT: A set of behavior specifications of
      malware, benign programs.

/* collect the execution traces of malware, benign file using
NtTrace */

3.  OutV[][0]←An Array containing OUT variable
/*OUT variable defined in NT internals of
Microsoft */
4.  InV[][0]←An Array containing IN variable /*IN
variable defined in NT internals of Microsoft */
5.  OutV[][1]←An Array containing System call
name having OUT variable
6.  InV[][1]←An Array containing System call name
having IN variable

/* Finding dependency between system calls when their
arguments are related */

7.  Scan the token from each system calls from the
beginning of the trace.
8.  While(!(token.equals("NtClose")))
9.  Begin
10. Find the value of Handle variable for each token
and assign these value in step 3, 4.
11. End
12. For(int k=0;k<OutV.length();k++)
13. For(int l=0;l<InV.length();l++)
14. Begin
15. If(OutV[k][0]!=null &&
InV[l][0]!=null)&&(OutV[k][0].equals(InV[l][0]))
16. Print ("There is a relation between"+
OutV[k][1]→InV[l][1]);
17. End
/*counting the Total dependency of a particular
System calls */
18. If(("OutV[k][1]→InV[l][1]").equals(s1→s2))
19. Begin
20. Countl++ /* frequency of sequence of system call
s1 and s2 */
21. End /* similarly we can apply for other relations
also */

```

Proposed algorithm for finding the behavior specifications

The above algorithm has been shown for finding the dependency between the system calls. We can also calculate the frequency of a particular system calls using this algorithm.

3.1.3 Static Analysis System for Malware Detection- Our static analysis system is based on the fact that every PE file has its own structural information [4],

this structural information is different for benign as well as malicious file. It has been observed experimentally that there is a false alarm in the case of packed benign, hence in our methodology we have separated the packed benign from unpacked benign using PEID tool. This has been observed that syntax of malware can be changed by obfuscation techniques such as polymorphic and metamorphic. Hence existing detectors fail to detect different variants of the same malware [3]. Additionally, static analysis is generally safer than dynamic analysis as the source code is not actually executed. The malware authors know the limitations of static analysis methods and thus, will likely create malware instances that employ these techniques to thwart static analysis. Therefore, it is necessary to develop analysis techniques that are resilient to such modifications, and are able to reliably analyze malicious software [3]. We have used PECOFF4J jar file to extract static features from the binary file.

IV. RESULTS AND DISCUSSION

Using Following experiments we have calculated the classification results of static and dynamic analysis approach.

Case 1-Static Analysis of Malware- In Table I, 15 features from a PE file have been extracted using PECOFF4J API. Feature selection process is then performed by selecting values that has significant difference as compare to benign file (shown by * in table I). Classification is done using J48 algorithm.

Table I. Static features that changes from benign to malicious.

Features	Benign(calc.exe)	Malicious(B1541.exe)
*No. of Sections	3	8
*Major Linker Version	9	8
*Minor Linker version	0	20
*Size of code	249856	42496
*Size of Initialized data	94208	28160
*Address of Entry point	413568	196609
Major Image Version	6	1
Minor Image Version	1	0
Size of Image	905216	208896
*DLL Characteristics	33088	0

Size of stack reserve	262144	2097152
Size of stack commit	131072	4096
Export table RVA	0	188416
*Export table Size	0	40
*Entropy	6.15	7.84

Static features of packed and unpacked file have been extracted using static analysis. The feature that changes from unpacked sample to packed sample significantly has been shown by * in Table II.

Table II. Static features that changes from unpacked benign to packed benign.

Features	Unpacked(calc.exe)	Packed(calc.exe)
File Size	776192	332800
*No of Section	4	3
Size of Code	339456	241664
*Size of Initialized Data	435712	94208
*Size of Uninitialized Data	0	552960
Address of Entry Point	77164	797568
*Base of Code	4096	557056
Base of data	335872	798720
Size of image	786432	892928
Import table RVA	334588	888292
*Import table Size	340	896
Resource RVA	364544	798720
*Resource Size	403352	89572
*Entropy	6.15	7.91

Case 2-Dynamic Analysis of malware

We have proposed an algorithm to calculate the system call dependency from the traces of malware. We have executed the malware in a controlled environment using NtTrace.

(a)*The Observed Behaviors among malicious files:* In Table III listed behaviors have been shown as a common run time behaviors of any malicious file. We have extracted the sequence of system calls representing these common behaviors in Table IV and have classified the malicious file based on these system calls.

(b)The Important system call's dependency that has been observed with high frequency among malware samples:

- NtEnumerateKey→NtOpenKey→NtQueryValueKey→NtDeviceControlFile{This represents leaking bug fix information over network }

- NtCreateKey→NtSetValueKey{this represents adding a new entry to the system auto start list}
- NtOpenKey→NtSetValueKey{this represents to bypass firewall to allow malicious traffic}

Table III. Malicious behaviors of Malware Files

Deleting a file.
Modifying a file.
Modifying a registry value.
Creating a registry key.
Attempts to replicate across the existing network(s).
Creating a process
Communication with a remote IRC server.
Blocking data
Copying data
Changing the browser proxy settings
Querying system information
Creating a file.

Table IV. Important system calls that have been found using proposed algorithms

*NtOpenSection→NtMapViewofSection
*NtOpenSection→NtClose
*NtCreateSection→NtClose
*NtOpenKey→NtQueryValueKey
NtOpenKey→NtClose
*NtOpenSection→NtQuerySection
*NtOpenProcessToken→NtQueryInformationToken
*NtOpenProcessToken→NtClose
*NtCreateSection→NtQuerySection
*NtOpenFile→NtClose
*NtCreateSection→NtMapViewofSection
NtCreateFile→NtClose
NtOpenFile→NtDeviceIoControlFile
*NtOpenFile→NtCreateSection
*NtCreateKey→NtSetValueKey
NtOpenKey→NtEnumerateValueKey
NtCreateEvent→NtDeviceIoControlFile

Fig 3 is representing the frequency of sequence of system calls. We are calculating the frequency of each sequence of System calls using proposed algorithm. We have considered only high frequency sequence of system calls for feature selection in our analysis system. The high frequency sequence of system calls have been shown by * in Table IV. We have taken 500 samples of malware including Backdoor, Trojan.win 32,Virus, Net worm, R-Bot, Z-Bot.

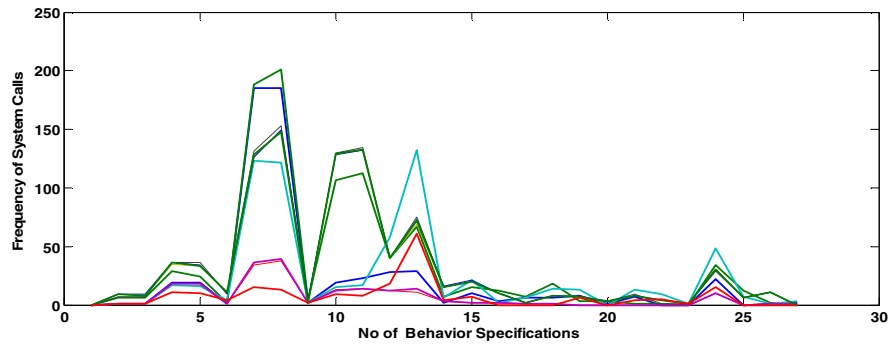


Fig. 3. System calls frequency Curve

Fig 4. is representing the TP and FP rate from proposed algorithm used for analysis of malware.

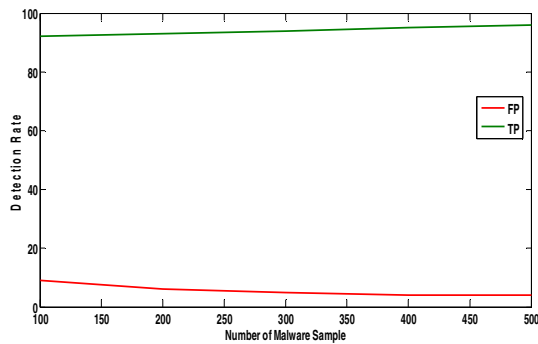


Fig. 4. TP and FP rate from proposed algorithm.

V. CONCLUSION

In this paper we have proposed a classification system for the detection of malicious executables using structural information and behavioral specification. We have also specified malware behaviors and implemented an algorithm for extracting such behavior specifications from the NtTrace of malware samples. It provides high detection accuracy up to 96% with low processing overhead. This work signifies the behavior differences between the benign and the malicious files. In future this work can be done by extracting the behavior specification through shared memory. In the same direction one can calculate the semantic based signature of malware. We expect that our approach will pave the way and inspire researchers in this area to achieve at more systematic methods for handling some issues related to the semantic-based approach in future.

REFERENCES

[1] Tian, Ronghua, Batten, Lynn, Islam, Rafiqul and Versteeg, "An automated classification system based on the strings of trojan and virus families", in

MALWARE : 4th International Conference on Malicious and Unwanted Software, IEEE, New York, N.Y., pp. 23-30,2009.

[2] S. Cesare, Y. Xiang, and W. Zhou, "Malwise: An effective and efficient classification system for packed and polymorphic malware," *Computers, IEEE Transactions on*, vol. 62, no. 6, pp. 1193–1206, 2013.

[3] S. Treadwell and M. Zhou, "A Heuristic Approach for Detection of Obfuscated Malware," in *Proceedings of the 3rd International Conference on Intelligence and Security Informatics*. IEEE, pp. 291–299, 2009.

[4] M.Z. Shafiq, S.M. Tabish, F. Mirza and M. Farooq, "PE-Miner: Mining Structural Information to Detect Malicious Executables in Realtime," *International Symposium On Recent Advances In Intrusion Detection (RAID)*, Lecture Notes in Computer Science, Springer, France, Vol. 5758, pp. 121-141, 2009.

[5] Xie, P.D., Lu, X.C., Su, J.S., Wang, Y.J., Li and M.J, "iPanda: A Comprehensive Malware Analysis Tool", In *27th International Conference on Information Networking (ICOIN'13)*. Bangkok, Thailand (2013).

[6] R. Lyda and J. Hamrock, "Using Entropy Analysis to Find Encrypted and Packed Malware," *IEEE Security and Privacy*, pp. 98-115, March/April 2007.

[7] T. Abou-Assaleh, N. Cercone, V. Keselj and R. Sweidan, "Ngram-based Detection of New Malicious Code," In *Proceedings of the 28th Annual International Computer Software and Applications Conference*, IEEE Computer, pp. 41-42, Society, 2004.

[8] H.J. Li, et al., "AOS: An optimized sandbox method used in behavior based malware detection," in the *International Conference on Machine Learning and Cybernetics (ICMLC'11)*, Vol. 1, pp. 404-409, 2011.

[9] Salehi Z., Ghiasi M. and Sami A. "A Miner for Malware Detection Based on API Function Calls and Their Arguments," in *16th CSI International Symposium on Artificial Intelligence and Signal Processing (AISP)*, Vol. 10, pp. 563- 568, 2012.

[10] C. Kolbitsch, P. M. Comparetti, C. Kruegel, E. Kirda, X. Zhou, and X. Wang, "Effective and efficient malware detection at the end host", In *USENIX Security Symposium*, Montreal, Canada, pp 38-49, 2009.

[11] M. Christodorescu, S. Jha, and C. Kruegel, "Mining specifications of malicious behavior," In *ESEC-FSE '07: Proceedings of the 6th joint meeting of the European software engineering conference and the ACM SIGSOFT symposium on The foundations of software engineering*, NewYork, USA, pp. 5–14, 2007.